

The Eight In-Video Questions

Exactly as the LMS receives them from Railway

Pulled live from `/api/v1/videos/<id>/checkpoints` on 2026-09-17, so this document cannot drift from what learners actually see.

Each video **pauses** at the time shown. The LMS presents the question, and **the learner must click an option to continue** – no skip, no dismiss, no seeking past it. They then see the feedback for their answer, and playback resumes at the same instant.

The correct option is marked. Both feedback texts are the real strings being served.

1 · Why a Chat Window Isn't Enough

pauses at 1:29 (89.34s)

Your agent answers correctly every time you open the project and type a prompt. Which of the three capabilities does it have?

- A Somewhere to track its work
- B A two-way line to you
- C A process running when you are not there
- D None of them

CORRECT

If they answer correctly: Correct — and that is why connecting one service never feels like enough. While the only way to reach it is to open the project, you are personally doing all three jobs.

If they answer wrongly: It has none of them, which surprises people because the setup clearly works. Look at what happens when you walk away: the task history disappears with the session, so there is no tracking; nothing can reach you, and you cannot reach it, without opening the editor, so there is no two-way line; and the process itself is your laptop, so nothing is running at all. Answering well while you watch is not the same as having any of the three.

videoId ns-01-why-a-chat-window · resumes at 89.34s · requiresAnswer true · allowSkip false · timing.trusted true · pause between beats 17→19

2 · The Loop, End to End

pauses at 1:02 (62.382s)

Your agent is connected to Slack, Notion and GitHub, and the loop works exactly as shown. You shut your laptop and go home. What happens to the next request someone sends?

- A It queues and runs when you open the laptop again
- B Nothing happens; there is no process running to receive it**
- C Slack retries until the agent responds
- D Notion creates the card and the rest waits

CORRECT

If they answer correctly: Exactly. Slack, Notion and GitHub are services the agent calls — none of them runs it. The code doing the reading, deciding and committing has to be executing somewhere, and if that somewhere is your laptop, closing it ends the loop.

If they answer wrongly: Nothing happens at all, and the reason it is tempting to think otherwise is that these services feel active. They are not. Slack delivers an event to a URL; if nothing answers, it retries briefly and gives up — it does not hold messages for an app that is switched off. Notion only creates a card when something calls its API, and nothing is calling. Your agent is a program, and a program that is not running cannot poll, cannot be notified, and cannot create anything.

videoId ns-02-the-loop · resumes at 62.382s · requiresAnswer true · allowSkip false · timing.trusted true · pause between beats 13→15

3 · Notion — The Task Board

pauses at 1:25 (85.63s)

You created the integration, copied the secret, and your code returns a 404 saying the database does not exist — but you are looking straight at it in your browser. What is wrong?

A The secret was copied incorrectly

B The database was never shared with the integration under Connections

CORRECT

C The database ID includes the view ID

D Notion needs a paid plan for API access

If they answer correctly: Exactly — and note how the error lies to you. It says "does not exist", which sends people hunting for a typo in the ID, when the real meaning is "exists, but not for you".

If they answer wrongly: It is the sharing step, and the reason it catches everyone is that you can see the database perfectly well — but you are signed in as you, and the integration is not you. It has its own identity, with access to nothing, until a human explicitly shares a page with it. A mistyped secret would fail as an authentication error, not a missing object; a view ID in the database ID gives a different and clearer error; and none of this requires a paid plan.

videoId ns-03-notion · resumes at 85.63s · requiresAnswer true · allowSkip false · timing.trusted true ·
pause between beats 16→18

4 · Slack — Speak, Then Listen

pauses at 1:30 (90.794s)

Your bot has chat:write, the app is installed, and the token is correct — but posting to your channel fails with not_in_channel. What is missing?

- A The channels:history scope
- B The bot was never invited to that channel **CORRECT**
- C The app needs reinstalling after adding scopes
- D The channel ID is wrong

If they answer correctly: Exactly — and the error name says it plainly once you read it as a statement of fact rather than a failure: the bot is not in the channel. Scopes say what it may do anywhere; membership says where.

If they answer wrongly: The bot is not a member of that channel. This is the most common Slack error, and it catches people because everything else is genuinely correct — the token works, the scope is granted, the app is installed. But a scope is a permission, not a presence: chat:write means "may send messages", not "is in this room". channels:history governs reading, not posting. Reinstalling matters when you add a scope, which fails differently (missing_scope). And a wrong channel ID gives channel_not_found — the error is telling you it found the channel fine and your bot simply is not in it.

videoId ns-04-slack · resumes at 90.794s · requiresAnswer true · allowSkip false · timing.trusted true · pause between beats 16→18

You added the webhook with a secret, deployed your server, and pushed an event — but the server rejects every delivery with a 403. What is almost certainly wrong?

- A The payload URL is wrong
- B The secret in GitHub and the secret on your server do not match **CORRECT**
- C The token needs Contents write permission
- D GitHub has not verified the webhook yet

If they answer correctly: Right — and this one is nasty because both halves look correct in isolation. GitHub shows a secret is set; your server has a secret set. Neither dashboard can tell you they are different strings.

If they answer wrongly: The two secrets do not match. The signature check recomputes the HMAC from the secret on your server and compares it to the header GitHub sent — if the strings differ by one character, or the environment variable was never actually set on the host, every delivery fails and returns 403. A wrong payload URL gives no delivery at all rather than a rejected one; the token's Contents permission governs committing, which has nothing to do with inbound verification; and GitHub does not hold webhooks back for verification the way Slack does.

videoId ns-05-github · resumes at 89.152s · requiresAnswer true · allowSkip false · timing.trusted true · pause between beats 18→20

6 · Your Deployment — The Body

pauses at 1:12 (72.909s)

You are setting up Slack's Event Subscriptions and you paste in `http://localhost:8000/slack/events`. Slack refuses to verify it. Why?

- A Slack requires HTTPS, so `https://localhost:8000` would work
- B Slack is calling your server from the internet and cannot reach an address that only exists on your laptop **CORRECT**
- C The endpoint path must be exactly `/slack/events`
- D Event Subscriptions must be toggled on before the URL is accepted

If they answer correctly: Exactly, and this is the concrete reason a deployment stops being optional the moment you want inbound events. Outbound calls work fine from a laptop — you are calling Slack. Inbound means Slack calls you.

If they answer wrongly: `localhost` means "this machine" — and the machine Slack is calling from is Slack's, not yours. Their servers resolve it to themselves, find nothing, and verification fails. Switching to `https` does not help, because the problem is reachability rather than the protocol; the path can be anything you like as long as your app serves it; and toggling Event Subscriptions is what produced the URL box in the first place. This is why you can post messages from your laptop all day but cannot receive a single one until the app lives at a public address.

videoId ns-06-deployment · resumes at 72.909s · requiresAnswer true · allowSkip false · timing.trusted true · pause between beats 14→16

7 · The Four Mistakes

pauses at 1:02 (62.279s)

Your agent posts to Slack perfectly, but has never once received a message. Which mistake is this?

- A The credential is set but the channel was never shared
- B The webhook points somewhere the outside world cannot reach **CORRECT**
- C The signing secret is set in only one place
- D A key was committed and has been revoked

If they answer correctly: Exactly — and the tell is the asymmetry. Sending working proves the token and scopes are fine, because you are the one calling Slack. Receiving nothing points at reachability.

If they answer wrongly: It is the reachability one, and the giveaway is that sending works. If the credential were unshared or the bot uninvited you could not post at all, so the thing that is working rules that out. A half-set signing secret would produce requests that arrive and get rejected — 403s in your logs — not silence. A revoked key would break sending too, loudly and immediately. Silence in one direction while the other is healthy nearly always means nothing is listening at a public address.

videoId ns-07-mistakes · resumes at 62.279s · requiresAnswer true · allowSkip false · timing.trusted true · pause between beats 12→14

Assignment · Connect Your Own Project

pauses at 1:30 (90.252s)

Your agent opens the pull request, but it could not read the Notion card back. So assignment.md lists a neat "To Do, In Progress, Done" with no timestamps — a sequence it worked out rather than observed. What should it have done instead?

- A Exactly that — the order is almost certainly right anyway
- B Left the status history out quietly and reported everything else
- C Said plainly that it could not retrieve the status history
- D Held the pull request back until it could read the card

CORRECT

If they answer correctly: Correct. The instruction is explicit: if any step fails, say so instead of skipping it. A named gap is useful information — it tells you exactly which permission is missing.

If they answer wrongly: It should say plainly that it could not retrieve the status history. This file exists for one reason: to show what actually happened, not what probably happened. A sequence the agent worked out looks identical on the page to one it observed, which is why a tidy history with no timestamps is marked as a failure rather than a formatting slip. Leaving it out quietly is the same problem in a quieter coat — the marker cannot tell a missing field from a missing permission. And holding the pull request back is worse than either: the work is finished, and it is the report that is incomplete.

videoId ns-assignment · resumes at 90.252s · requiresAnswer true · allowSkip false · timing.trusted true · pause between beats 12→14